

Laboratorio #5 — Agentes en el Arcade Learning Environment: Space Invaders

CC3092 Deep Learning y Sistemas Inteligentes

14 de septiembre de 2026

1. El Arcade Learning Environment

ALE expone los juegos de Atari 2600 como entornos de aprendizaje por refuerzo. Resuelve un problema de **evaluación**: antes de ALE cada investigador probaba su algoritmo en un dominio hecho a medida y los resultados no eran comparables. ALE ofrece más de 100 juegos con una interfaz idéntica (imagen de 210×160 y unas pocas acciones de joystick), diseñados por terceros y no por investigadores de IA, lo que evita sesgar el benchmark.

ALE no emula nada por sí mismo: es una capa sobre **Stella**, un emulador de Atari 2600 de código abierto. Stella ejecuta la ROM original del juego; ALE se conecta a él para leer la pantalla y los 128 bytes de RAM, inyectar acciones de joystick, y extraer del estado interno el *score* y las vidas, que traduce a la señal de recompensa. Las ROMs son las originales sin modificar y desde `ale-py` 0.10 vienen incluidas en el paquete.

Variantes del mismo juego. Un juego aparece bajo varios IDs cuyas diferencias cambian la dificultad y la comparabilidad:

Variante	frameskip	repeat_action_probability	Notas
SpaceInvaders-v0	(2,5) aleatorio	0.25	legacy
SpaceInvaders-v4	(2,5) aleatorio	0.0	legacy, determinista
SpaceInvadersNoFrameskip-v4	1	0.0	el del paper de DQN, con wrappers propios
ALE/SpaceInvaders-v5	4	0.25	recomendado , sigue Machado et al. (2018)

`repeat_action_probability` (*sticky actions*) hace que con probabilidad 0.25 el emulador ignore la acción nueva y repita la anterior; es la forma estándar de inyectar estocasticidad, porque sin ella un agente puede memorizar una secuencia fija de acciones y obtener un *score* altísimo sin haber aprendido nada. `full_action_space=True` expone las 18 acciones del joystick para todos los juegos, útil para entrenar un solo agente sobre muchos juegos a la vez. El sufijo `-ram` no es un ID en `ale-py` 0.12 sino el argumento `obs_type="ram"`.

Frame skipping. Con `frameskip=k` cada `step()` repite la misma acción durante *k* frames y devuelve sólo el último (o el máximo píxel a píxel de los dos últimos, para eliminar el parpadeo de sprites del hardware); la recompensa devuelta es la suma de los *k* frames. Importa por dos razones. En **velocidad**: con *k*=4 el agente decide 4 veces menos y, como el agente es la parte cara del pipeline, es la mayor ganancia de rendimiento disponible. En **aprendizaje**: dos frames consecutivos a 60 Hz son casi idénticos, así que decidir en cada uno es redundante; saltar frames acorta el horizonte efectivo del episodio y facilita la asignación de crédito. El precio es la granularidad de reacción.

Space Invaders. El jugador controla un cañón que se mueve horizontalmente y dispara hacia arriba contra una formación de alienígenas que desciende disparando; hay escudos destructibles y una nave nodriza que vale más puntos. Se pierde una de 3 vidas al ser alcanzado. ALE lee el *score* de la RAM y define la recompensa de cada paso como el **incremento de score** ($r_t = \text{score}_t - \text{score}_{t-1}$): 0 en la mayoría de pasos, positiva al destruir un alienígena (10–30 puntos) o la nave nodriza (50–200). El retorno del episodio es el *score* final. Es una señal **dispersa** y nunca negativa: perder una vida no resta puntos, sólo acorta el episodio.

2. Espacios de observación y acción

ALE/SpaceInvaders-v5 observa `Box(0, 255, (210,160,3), uint8)` — la pantalla RGB cruda, 100 800 valores — frente al `Box(4,)` de `float32` de `CartPole-v1`, que son las cuatro variables físicas [*x*, *ẋ*, *θ*, *θ̇*] con significado directo. Trabajar con imágenes cambia cuatro cosas: (1) hace falta una **CNN**, porque un MLP sobre 100 800 entradas es inviable, así que el problema deja de ser sólo de control y pasa a ser también de visión; (2) la observación **deja de ser el estado** — un frame estático no dice hacia dónde se mueve una bala — de modo que el proceso es un POMDP y no un MDP sobre la observación; (3) el **costo de memoria** explota: un replay buffer de 1M de transiciones sin preprocesar serían ~200 GB; y (4) los valores son `uint8` en [0,255] y hay que normalizarlos antes de la red.

La variante `obs_type="ram"` devuelve `Box(0, 255, (128,), uint8)`: los 128 bytes de memoria de la consola, donde viven la posición del cañón, la de cada alienígena, el *score* y las vidas. Conviene cuando interesa el problema de control y no el de visión — se evita entrenar una CNN y los experimentos corren en minutos —, como límite superior de referencia para medir cuánto pierde el agente basado en píxeles, o cuando hay poco cómputo. Sus límites: la codificación es opaca y distinta en cada ROM, así que no transfiere entre juegos ni es comparable con la literatura, que reporta sobre píxeles.

El espacio de acción es `Discrete(6)`, el subconjunto mínimo que el juego usa:

Índice	Acción	Efecto
0	NOOP	no hacer nada

Índice	Acción	Efecto
1	FIRE	disparar sin moverse
2 / 3	RIGHT / LEFT	mover sin disparar
4 / 5	RIGHTFIRE / LEFTFIRE	mover y disparar

Son combinaciones de dirección × botón del joystick. Las compuestas (4 y 5) son las útiles en la práctica: permiten esquivar sin dejar de disparar.

AtariPreprocessing reproduce el preprocesamiento del paper de DQN: escala de grises (el color no aporta información en Atari), redimensionado a 84×84 (100 800 → 7 056 valores), *frame skipping* con max-pooling de los 2 últimos frames, `noop_max=30` para que los episodios no arranquen siempre igual, `terminal_on_life_loss`, `scale_obs` a `float32` en `[0,1]`, y recorte de recompensa a `{-1, 0, +1}` para poder usar los mismos hiperparámetros en juegos con escalas de score muy distintas. `FrameStackObservation` apila las últimas `k=4` observaciones en un tensor (4, 84, 84); se usa junto con el anterior porque resuelve exactamente el problema (2) de arriba: con 4 frames consecutivos la red puede inferir velocidad y dirección, y la observación vuelve a aproximar un estado markoviano. El orden importa: primero preprocesar, después apilar. En este laboratorio ambos se documentan pero **no se aplican**, porque no se entrena nada y el video debe grabarse con la imagen original.

3. Módulo de desarrollo y resultados

`ale_utils.py` contiene cinco funciones con las firmas del enunciado. El notebook lo importa; no redefine nada.

Función	Qué hace
<code>crear_entorno(nombre_entorno, video_folder=None, episode_trigger=None, ...)</code>	crea el entorno y, si se da <code>video_folder</code> , lo envuelve con <code>RecordVideo</code>
<code>agente_aleatorio(observation, env)</code>	<code>baseline: env.action_space.sample()</code>
<code>agente_regla_simple(observation, env)</code>	política fija no aprendida: barre de lado a lado disparando
<code>ejecutar_episodio(env, funcion_agente, max_steps=10000)</code>	corre un episodio; retorna (pasos, recompensa_total)
<code>generar_video_agente(nombre_entorno, funcion_agente, video_folder, name_prefix, n_episodios=1)</code>	combina las anteriores y cierra el entorno

Las funciones son genéricas: la misma llamada sirve para `ALE/SpaceInvaders-v5` y para `CartPole-v1`. Cuatro detalles no son opcionales y cada uno costó una corrida fallida:

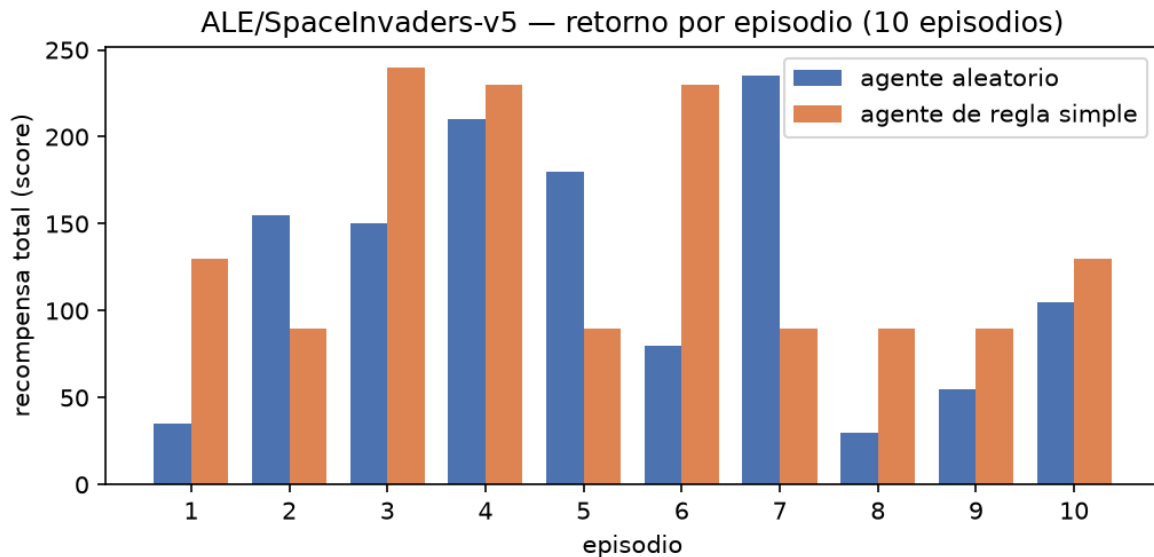
- `gym.register_envs(ale_py)` al importar el módulo; sin eso `gym.make("ALE/SpaceInvaders-v5")` lanza `NameNotFound` desde `ale-py 0.10`.
- `env.close()` en un `finally`: `RecordVideo` escribe el archivo al cerrar el entorno, así que sin `close()` el último `.mp4` queda vacío.
- `env.reset(seed=...)` **no** siembra el RNG de `env.action_space`, de modo que `agente_aleatorio` daba números distintos en cada corrida hasta llamar explícitamente a `env.action_space.seed(seed)`.
- `agente_regla_simple` lleva un contador de pasos; si no se reinicia por episodio el resultado depende de cuántas veces se llamó antes al agente (el mismo experimento daba 141 o 207 según si se habían grabado videos primero). `ejecutar_episodio` llama a `funcion_agente.reset()` cuando el agente expone ese atributo.

Videos generados (videos/, un episodio completo cada uno, semilla 0):

Video	Agente	Pasos sobrevividos	Recompensa total
<code>spaceinvaders_random-episode-0.mp4</code>	aleatorio	298	35.0
<code>spaceinvaders_regla-episode-0.mp4</code>	regla simple	405	130.0

Comparación sobre 10 episodios (semillas 0–9, sin grabar):

Agente	Retorno medio	Desv. est.	Mín.	Máx.	Pasos medios
aleatorio	123.5	69.5	30	235	461.0
regla simple	141.0	62.4	90	240	406.7



Retorno por episodio

La regla simple le gana al azar por ~14%, pero con desviaciones de ese tamaño y sólo 10 episodios la diferencia no es concluyente. Lo que sí es claro es que la regla **sobrevive menos** (406.7 contra 461.0 pasos) y aun así puntea más: no juega mejor a la defensiva, sólo dispara más por unidad de tiempo. Tres razones acotan la mejora: las *sticky actions* degradan el barrido “determinista”; la regla no mira la pantalla, así que no esquiva nada y muere antes; y en Space Invaders sólo puede haber un disparo del jugador en pantalla a la vez, de modo que pulsar FIRE siempre no sube la cadencia por encima de ese límite. Eso es justamente lo que motiva el aprendizaje: la política tiene que ser **función de la observación**.

El agente aleatorio no es relleno: es la línea base contra la que se mide cualquier agente entrenado. ~124 puntos es el “cero” de este juego, frente a los ~1 700 que reporta el DQN original. Para entrenar no hay que tocar estas funciones: basta envolver el entorno con `AtariPreprocessing + FrameStackObservation` dentro de `crear_entorno` y que `funcion_agente` pase de ser una función pura al `forward` de una CNN. La firma `funcion_agente(observation, env) -> action` ya soporta eso sin cambios.

4. Reglas de tecnología

El objetivo del lab es la infraestructura, no entrenar un agente. De ahí las restricciones (versión completa y justificada en `CLAUDE.md` del directorio):

Permitido	Python 3.10+, gymnasium 1.3.0, ale-py ≥0.12, numpy, matplotlib, moviepy/imageio-ffmpeg sólo como backend de RecordVideo, nbformat/nbconvert para el notebook, stdlib
Entrenar agentes	prohibido: Q-learning, DQN, policy gradient, replay buffer, ϵ -greedy con decaimiento, redes neuronales, checkpoints. Sólo agente aleatorio y regla fija no aprendida
torch, tensorflow, keras, jax	prohibidos: no hay modelo que entrenar
stable-baselines3, ray[rllib], tianshou, cleanrl	prohibidos: sustituyen la infraestructura que es el objeto del lab
gym legacy (OpenAI Gym)	prohibido: mezclar APIs rompe el contrato de 5 valores de <code>step()</code>
atari-py, AutoROM	prohibidos: obsoletos, ale-py ≥0.10 ya trae las ROMs
opencv, Pillow para preprocesar frames a mano	prohibidos: para eso están los wrappers de Gymnasium
escribir mp4 a mano (imageio.mimwrite, ffmpeg por subprocess)	prohibido: el enunciado exige <code>gymnasium.wrappers.RecordVideo</code>
clases, factories o capas de configuración sobre las 5 funciones	prohibidas: deben quedar como funciones sueltas con las firmas del enunciado
<code>render_mode="human"</code> o dependencias de display	prohibidos: grabar requiere <code>rgb_array</code> y el lab debe correr <code>headless</code>

Verificación: `python ale_utils.py` corre un self-check con asserts (espacios del entorno, acciones válidas, genericidad sobre `CartPole-v1`, `max_steps`, independencia del historial de llamadas, y que el `.mp4` generado no quede vacío) e imprime `self-check OK`.